

AI Engineering Worked Examples and Reference

Worked examples that connect prompting, retrieval, memory, tools, FastAPI, SQLite, and CI.

Engineering notes and worked references

Contents

1. Worked example: strict JSON lesson result
2. Worked example: embedding shape
3. Worked example: chunk boundary
4. Worked example: RAG trace
5. Worked example: vague follow-up
6. Worked example: tool call and observation
7. Worked example: router decision
8. Worked example: FastAPI validation
9. Worked example: session persistence
10. Worked example: CI catches a regression
11. Worked example: unsupported question

1. Worked example: strict JSON lesson result

A lesson service asks the model for JSON containing title, learning_objectives, lesson_sections, practice, and quiz.

The response is parsed and then checked before it is saved. If quiz is missing, the application should not store the object as a complete lesson.

Example from use

Input topic: Python loops, level: beginner, duration: 60 minutes.

The validator checks both required keys and expected value types.

Small code example

```
raw = model_response_text
lesson = json.loads(raw)

if not isinstance(lesson["learning_objectives"], list):
    raise ValueError("learning_objectives must be a list")
```

Things people mix up

- Saving before validation.
- Checking only whether the response starts with '{'.
- Letting the model invent the database ID.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

What to look at while running it

In the worked example: strict json lesson result example, do not stop at the final successful output. Inspect the request or input, one useful intermediate value, and the final stored or returned result. That makes the example useful for debugging later.

A live demo is stronger when the same case can be run twice and gives behavior you can explain. If the result changes because an external model generates slightly different wording, the structural parts such as source IDs, database rows, route status, and validation should still be understandable.

Small variation to try

- Change one input value.
- Run a second session or source query.
- Introduce one controlled error.
- Explain which layer should catch it.

The point of the variation is not to make the example harder. It is to show that the implementation is understood well enough to predict what should change.

2. Worked example: embedding shape

A notes folder contains 240 chunks and uses all-MiniLM-L6-v2. Each chunk becomes a 384-dimensional vector.

The embedding matrix has 240 rows and 384 columns. A new question is embedded into one 384-dimensional query vector and compared with the stored rows.

Example from use

The vector length must match the index dimension.

The original chunk text and chunk ID remain stored for source display.

Reference table

Object	Shape
240 stored chunks	240 x 384
1 query	1 x 384
top-5 result IDs	5 identifiers plus scores

Things people mix up

- Calling 240 the embedding dimension.
- Discarding chunk text after indexing.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

What to look at while running it

In the worked example: embedding shape example, do not stop at the final successful output. Inspect the request or input, one useful intermediate value, and the final stored or returned result. That makes the example useful for debugging later.

A live demo is stronger when the same case can be run twice and gives behavior you can explain. If the result changes because an external model generates slightly different wording, the structural parts such as source IDs, database rows, route status, and validation should still be understandable.

Small variation to try

- Change one input value.
- Run a second session or source query.
- Introduce one controlled error.
- Explain which layer should catch it.

The point of the variation is not to make the example harder. It is to show that the implementation is understood well enough to predict what should change.

3. Worked example: chunk boundary

A source section contains a definition, an example, and a warning that changes how the definition should be interpreted.

If fixed chunking separates the warning from the definition, retrieval may return an incomplete rule. Keeping the section together or adding modest overlap can preserve the needed context.

Example from use

Chunk A ends with 'Enterprise supports up to 24 months'. Chunk B starts with 'No standard plan has unlimited history.' A question about unlimited history may need both.

Section-aware chunking can keep these sentences together.

Things people mix up

- Increasing top-k without noticing the boundary problem.
- Making every chunk huge enough to contain the whole document.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

What to look at while running it

In the worked example: chunk boundary example, do not stop at the final successful output. Inspect the request or input, one useful intermediate value, and the final stored or returned result. That makes the example useful for debugging later.

A live demo is stronger when the same case can be run twice and gives behavior you can explain. If the result changes because an external model generates slightly different wording, the structural parts such as source IDs, database rows, route status, and validation should still be understandable.

Small variation to try

- Change one input value.
- Run a second session or source query.
- Introduce one controlled error.
- Explain which layer should catch it.

The point of the variation is not to make the example harder. It is to show that the implementation is understood well enough to predict what should change.

4. Worked example: RAG trace

A user asks, 'Why can increasing top-k make an answer worse?' The search query is already clear, so no rewrite is needed.

The retriever returns the top-k section and a debugging section. The answer uses those passages to explain that extra weak context can add noise and consume prompt space.

Example from use

Original query: Why can increasing top-k make an answer worse?

Retrieved: vec-08 score 0.83, rag-10 score 0.74.

Final sources: vector search notes / Top-k; RAG notes / Evaluation.

Things people mix up

- Returning an unrelated memory chunk because it mentions context.
- Adding a claim that top-k above 5 is always wrong.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

What to look at while running it

In the worked example: rag trace example, do not stop at the final successful output. Inspect the request or input, one useful intermediate value, and the final stored or returned result. That makes the example useful for debugging later.

A live demo is stronger when the same case can be run twice and gives behavior you can explain. If the result changes because an external model generates slightly different wording, the structural parts such as source IDs, database rows, route status, and validation should still be understandable.

Small variation to try

- Change one input value.
- Run a second session or source query.
- Introduce one controlled error.
- Explain which layer should catch it.

The point of the variation is not to make the example harder. It is to show that the implementation is understood well enough to predict what should change.

5. Worked example: vague follow-up

Earlier turns discuss vector search. The user then asks, 'what about top-k?'. The raw message is understandable to a human reading the conversation but weak as a standalone vector query.

The application rewrites it to 'How does top-k affect vector search result selection?' and uses that for retrieval. The answer shown to the user is still a normal explanation, not the rewritten sentence.

Example from use

History provides the subject. The rewrite only resolves the missing reference.

A new unrelated message should bypass this rewrite.

Things people mix up

- Embedding the raw phrase only.
- Putting the whole history into the rewritten query.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

What to look at while running it

In the worked example: vague follow-up example, do not stop at the final successful output. Inspect the request or input, one useful intermediate value, and the final stored or returned result. That makes the example useful for debugging later.

A live demo is stronger when the same case can be run twice and gives behavior you can explain. If the result changes because an external model generates slightly different wording, the structural parts such as source IDs, database rows, route status, and validation should still be understandable.

Small variation to try

- Change one input value.
- Run a second session or source query.
- Introduce one controlled error.
- Explain which layer should catch it.

The point of the variation is not to make the example harder. It is to show that the implementation is understood well enough to predict what should change.

6. Worked example: tool call and observation

The user says, 'Save that explanation as Top-k notes.' The model can request a `save_note` action with title and content.

The application validates the arguments, supplies the trusted session ID, inserts the record into SQLite, and returns the new note ID as the tool result.

Example from use

Tool request: `save_note(title='Top-k notes', content='...')`.

Observation: `{'note_id': 'N-104', 'saved': True}`.

Small code example

```
note_id = repo.save_note(
    session_id=current_session,
    title=args.title,
    content=args.content
)
```

Things people mix up

- Letting the model choose another user's session ID.
- Claiming success before the insert commits.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

What to look at while running it

In the worked example: tool call and observation example, do not stop at the final successful output. Inspect the request or input, one useful intermediate value, and the final stored or returned result. That makes the example useful for debugging later.

A live demo is stronger when the same case can be run twice and gives behavior you can explain. If the result changes because an external model generates slightly different wording, the structural parts such as source IDs, database rows,

route status, and validation should still be understandable.

Small variation to try

- Change one input value.
- Run a second session or source query.
- Introduce one controlled error.
- Explain which layer should catch it.

The point of the variation is not to make the example harder. It is to show that the implementation is understood well enough to predict what should change.

7. Worked example: router decision

A router has three simple paths: direct response, document search, and stored-note action.

'Hello' uses direct response. 'What is cosine similarity?' uses document search. 'List my saved notes' uses the stored-note action.

Example from use

The route follows intent, not message length.

A request can later use more than one path if it genuinely needs multiple steps.

Reference table

User message	Route
Hello	direct
What is cosine similarity?	document search
List my saved notes	saved-note action

Things people mix up

- Running retrieval for greetings.
- Sending every message containing a number to a calculator.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

What to look at while running it

In the worked example: router decision example, do not stop at the final successful output. Inspect the request or input, one useful intermediate value, and the final stored or returned result. That makes the example useful for debugging later.

A live demo is stronger when the same case can be run twice and gives behavior you can explain. If the result changes because an external model generates slightly different wording, the structural parts such as source IDs, database rows, route status, and validation should still be understandable.

Small variation to try

- Change one input value.
- Run a second session or source query.
- Introduce one controlled error.
- Explain which layer should catch it.

The point of the variation is not to make the example harder. It is to show that the implementation is understood well enough to predict what should change.

8. Worked example: FastAPI validation

POST /chat expects session_id and message. A caller sends only session_id.

Pydantic rejects the request before the conversation pipeline runs. The API returns a validation error instead of creating an empty message.

Example from use

This is expected API behavior, not an LLM failure.

The route test should confirm the model client was not called.

Small code example

```
class ChatRequest(BaseModel):
    session_id: str
    message: str
```

Things people mix up

- Making message optional to hide the error.
- Returning a custom 500 from inside the route.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

What to look at while running it

In the worked example: fastapi validation example, do not stop at the final successful output. Inspect the request or input, one useful intermediate value, and the final stored or returned result. That makes the example useful for debugging later.

A live demo is stronger when the same case can be run twice and gives behavior you can explain. If the result changes because an external model generates slightly different wording, the structural parts such as source IDs, database rows, route status, and validation should still be understandable.

Small variation to try

- Change one input value.
- Run a second session or source query.
- Introduce one controlled error.
- Explain which layer should catch it.

The point of the variation is not to make the example harder. It is to show that the implementation is understood well enough to predict what should change.

9. Worked example: session persistence

Session research-001 asks about embeddings and saves a note. Session research-002 asks about FastAPI. The API process is restarted.

After restart, research-001 still loads its own note and research-002 still has separate history because both were stored in SQLite with session IDs.

Example from use

Run two interleaved requests before restart.

Query the database to verify messages belong to the correct session.

Things people mix up

- Using one global history list.
- Testing only one session.
- Using an in-memory SQLite database.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

What to look at while running it

In the worked example: session persistence example, do not stop at the final successful output. Inspect the request or input, one useful intermediate value, and the final stored or returned result. That makes the example useful for debugging later.

A live demo is stronger when the same case can be run twice and gives behavior you can explain. If the result changes because an external model generates slightly different wording, the structural parts such as source IDs, database rows, route status, and validation should still be understandable.

Small variation to try

- Change one input value.
- Run a second session or source query.
- Introduce one controlled error.
- Explain which layer should catch it.

The point of the variation is not to make the example harder. It is to show that the implementation is understood well enough to predict what should change.

10. Worked example: CI catches a regression

A test verifies that `list_notes` only returns notes for the requested session. A code change accidentally removes the session filter.

The GitHub Actions job runs `pytest` on push. The session-isolation test fails, the workflow turns red, and the bug is fixed before merge.

Example from use

The failing check demonstrates that CI is connected to real behavior.

The final commit restores the WHERE session_id condition and the job passes.

Things people mix up

- Ignoring the red check because manual testing looked fine.
- Changing the test to match the broken behavior.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

What to look at while running it

In the worked example: ci catches a regression example, do not stop at the final successful output. Inspect the request or input, one useful intermediate value, and the final stored or returned result. That makes the example useful for debugging later.

A live demo is stronger when the same case can be run twice and gives behavior you can explain. If the result changes because an external model generates slightly different wording, the structural parts such as source IDs, database rows, route status, and validation should still be understandable.

Small variation to try

- Change one input value.
- Run a second session or source query.
- Introduce one controlled error.
- Explain which layer should catch it.

The point of the variation is not to make the example harder. It is to show that the implementation is understood well enough to predict what should change.

11. Worked example: unsupported question

The source library covers prompting, embeddings, RAG, memory, tools, FastAPI, SQLite, Git, and CI. A user asks for the details of a Kubernetes admission controller feature that is not described in the notes.

The assistant should say the supplied notes do not cover that topic. It can mention that the library includes deployment-adjacent material only if that is actually present, but it should not silently answer from general model knowledge.

Example from use

A weak semantic match to 'CI' is not enough support for the Kubernetes question.

The response can ask whether the user wants an answer using outside knowledge, if the product allows that mode.

Things people mix up

- Giving a confident general-knowledge answer anyway.
- Inventing a source citation.

- Saying 'I don't know' without checking the whole note library first.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

What to look at while running it

In the worked example: unsupported question example, do not stop at the final successful output. Inspect the request or input, one useful intermediate value, and the final stored or returned result. That makes the example useful for debugging later.

A live demo is stronger when the same case can be run twice and gives behavior you can explain. If the result changes because an external model generates slightly different wording, the structural parts such as source IDs, database rows, route status, and validation should still be understandable.

Small variation to try

- Change one input value.
- Run a second session or source query.
- Introduce one controlled error.
- Explain which layer should catch it.

The point of the variation is not to make the example harder. It is to show that the implementation is understood well enough to predict what should change.